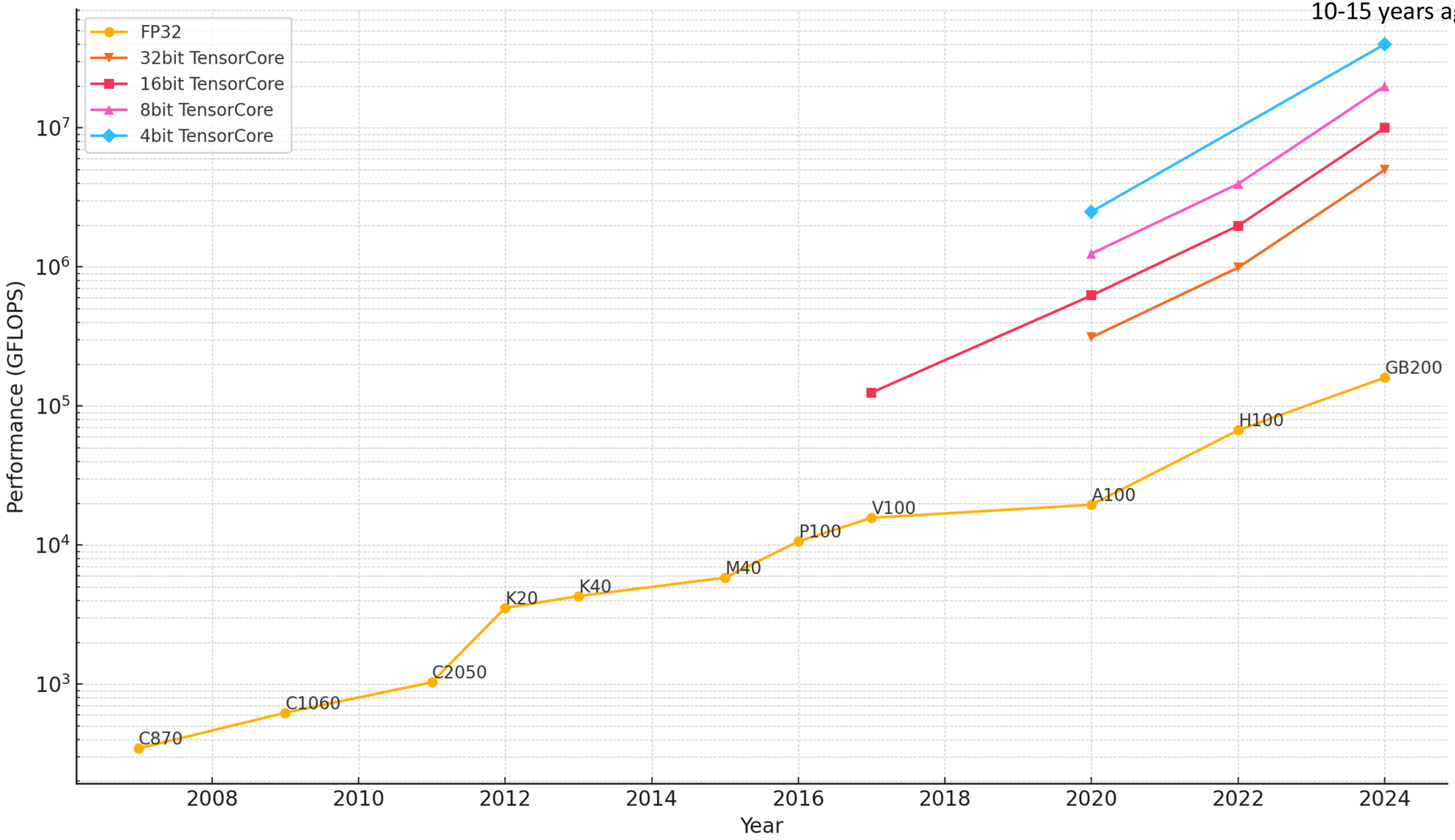
GPU-Accelerated EDA: Challenges and Opportunities

Haoxing (Mark) Ren

Ever Increasing GPU Performance

Over 1000X in last 10 years

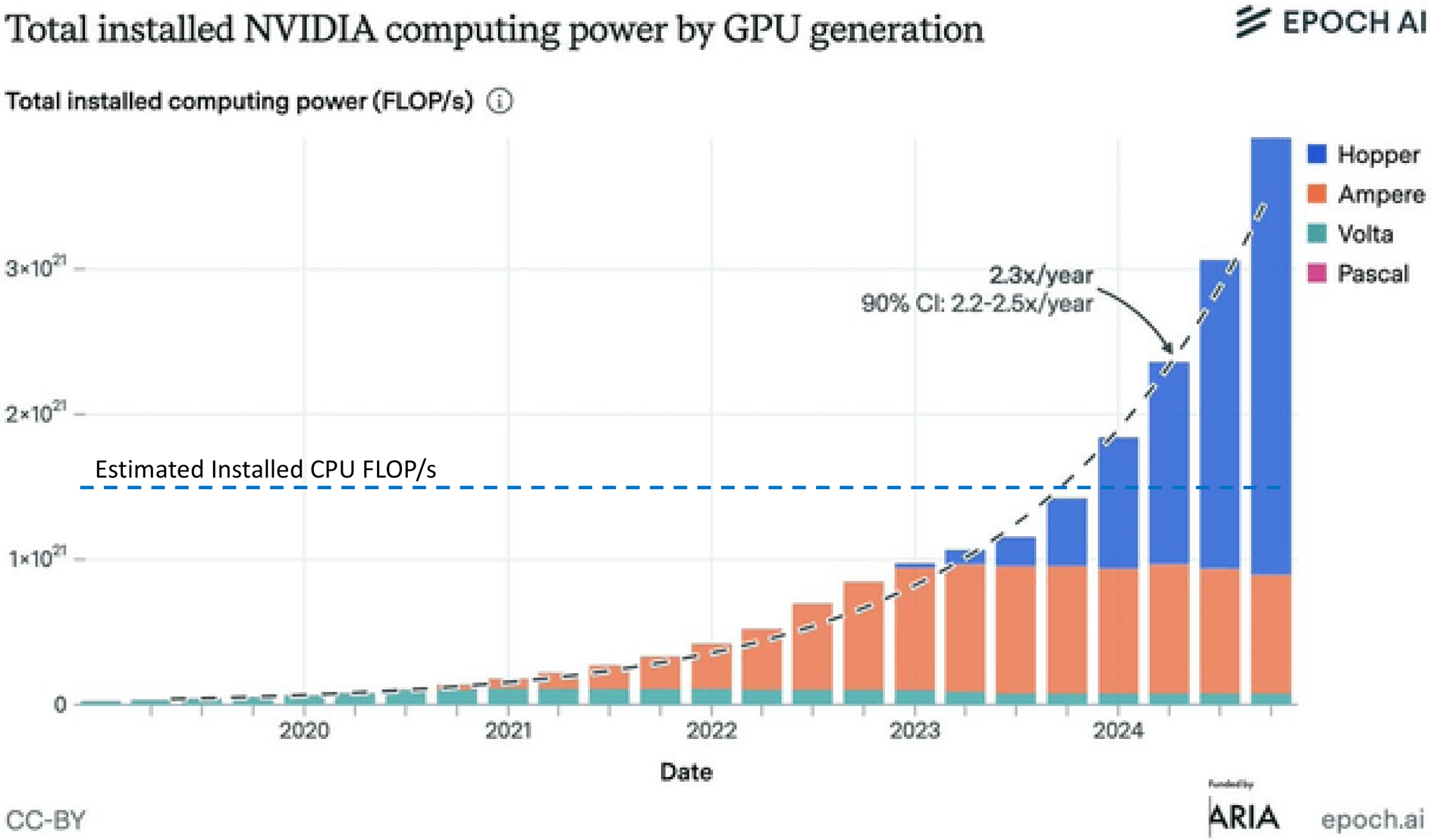
Single chip performance
Equivalent to Supercomputing
10-15 years ago



NVIDIA

Rapids Global GPU Adoption

Estimated installed GPU compute doubling every 10 months, dominating total computing capacity



GPU-Accelerated EDA Challenges

Why has the GPU not become mainstream in EDA?

ROI of investing in GPU-acceleration is low ?

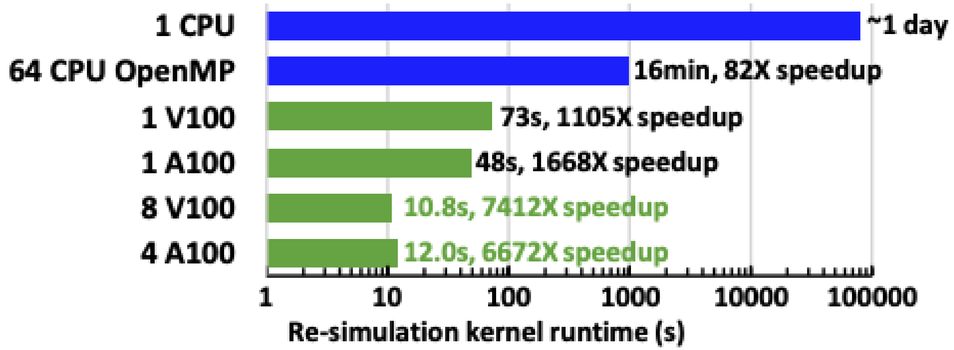
- Limits of parallelism- EDA problems are not ‘embarrassingly parallel’
- Performance limitation – divergence, communication, and synchronization overheads
- Algorithm scalability – EDA algorithms are difficult to vectorize to scale on GPU
- Discrete problems – EDA problems are discrete and difficult to scale
- Limits of scaling – Amdahl’s Law
- Cost of development - CUDA code and maintaining two different versions of the tools

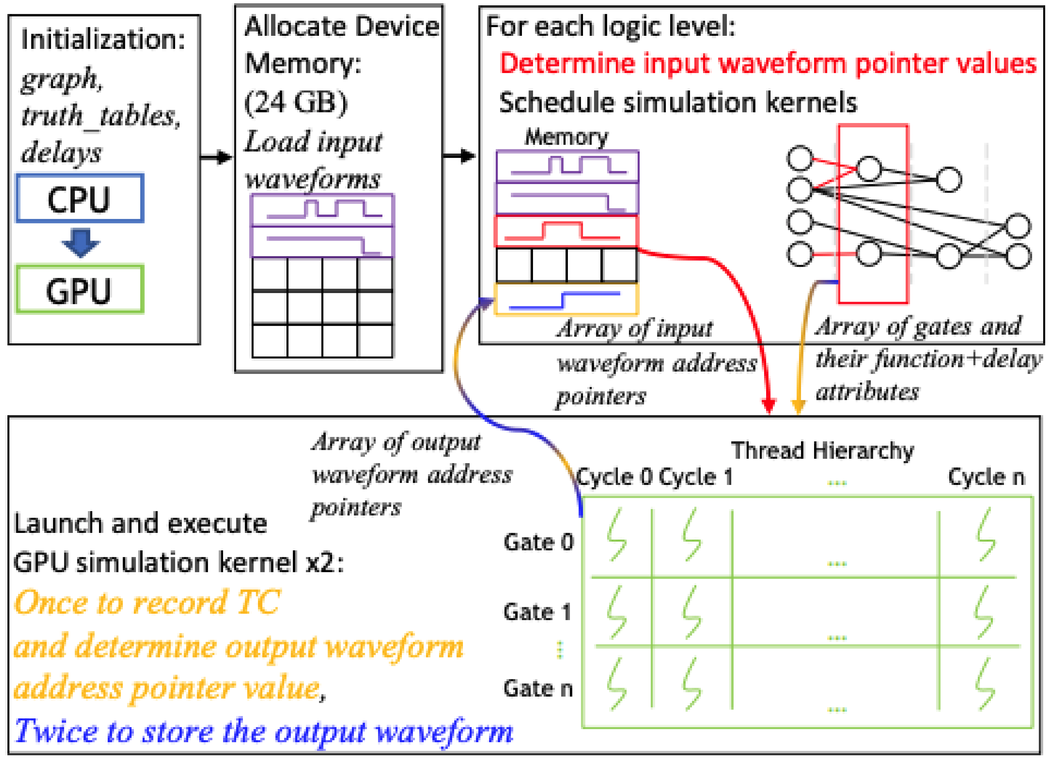
GATSPI - GPU Accelerated Gate-Level Simulator

Create an 'embarrassingly parallel' problem by leveraging cycle-level parallelism

- Goal: Gate-level re-simulation for performance analysis: power
- Known multi-cycle waveform at sequential elements and inputs
- Cycle-level parallelism and Gate-level parallelism
- Each thread simulates a gate on a cycle
- Used in production power analysis within NVIDIA

Design(Testbench)	T4	V100	A100
NVDLA,large(scan)	42.53(60X)	9.99(254X)	6.59(385X)
Design B(func. 2)	76.56(195X)	14.55(1026X)	12.11(1232X)
Design B(high activity)	260.15(179X)	38.90(1198X)	25.50(1828X)



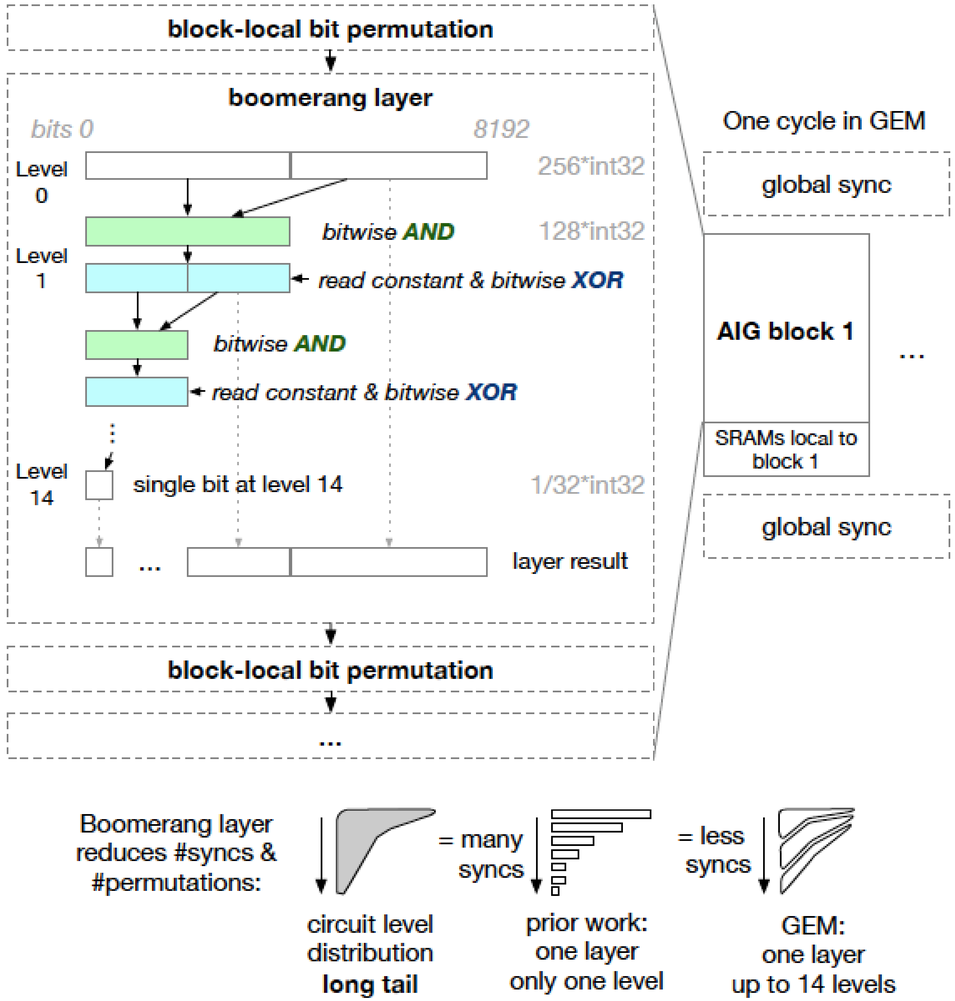


Y. Zhang et al, GATSPI: GPU Accelerated Gate-Level Simulation for Power Improvement, DAC'22

GEM: GPU-Accelerated Emulator-Inspired RTL Simulation

Leverage compiler to reduce divergence,, communication and synchronization overheads

- Convert RTL into AIG, execute AIG boolean logic in VLIW fashion
- Move from bit-level to word-level instructions with coalesced access
- Each thread block computes an independent partition, and global synchronization only needed cross partitions
- Compilation and partition flow the key to enable coalesced access and reduced synchronization between stages
- Up to 64X speedup over Verilator and 38x over commercial tools
- Low memory consumption and just as effective on 3090



Z. Guo et al, GEM: GPU-Accelerated Emulator-Inspired RTL Simulation, DAC'25

Accelerate Placement on GPU

Choose GPU-scalable algorithm: gradient-based numerical optimization

Minimize wirelength and density overflow

$$\min \quad f = WL(x,y) + \mu \sum_{bin} Overflow_{bin}(x,y)^2$$

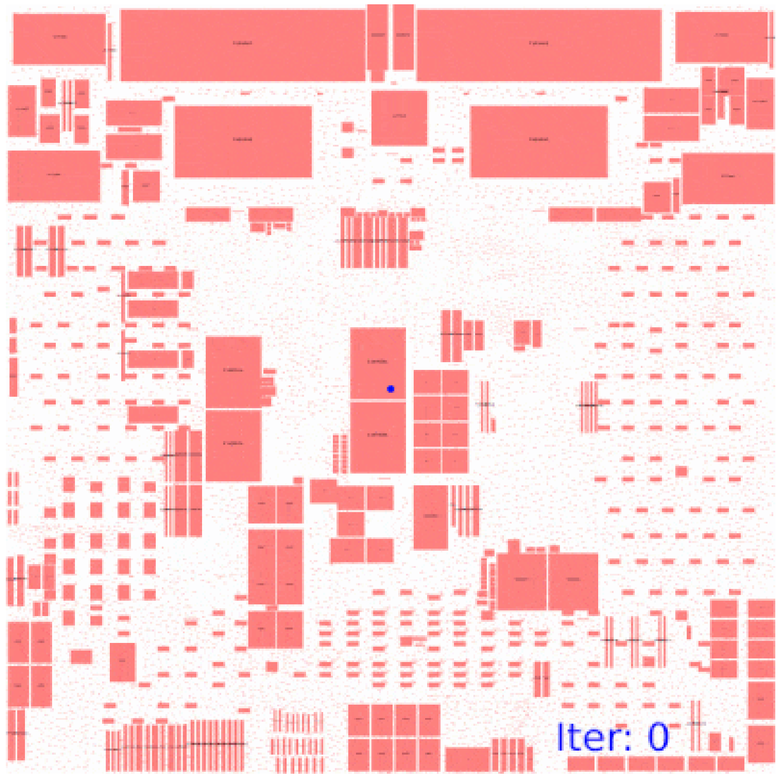
Gradient-based optimization :

$$x = x - \gamma \frac{\partial}{\partial x} f$$

$$y = y - \gamma \frac{\partial}{\partial y} f$$

Neural networks back propagation

$$\min \quad \sum_{i} loss(w, x_i)^2 + L_2(w)$$



Accelerate with DL Frameworks on GPU

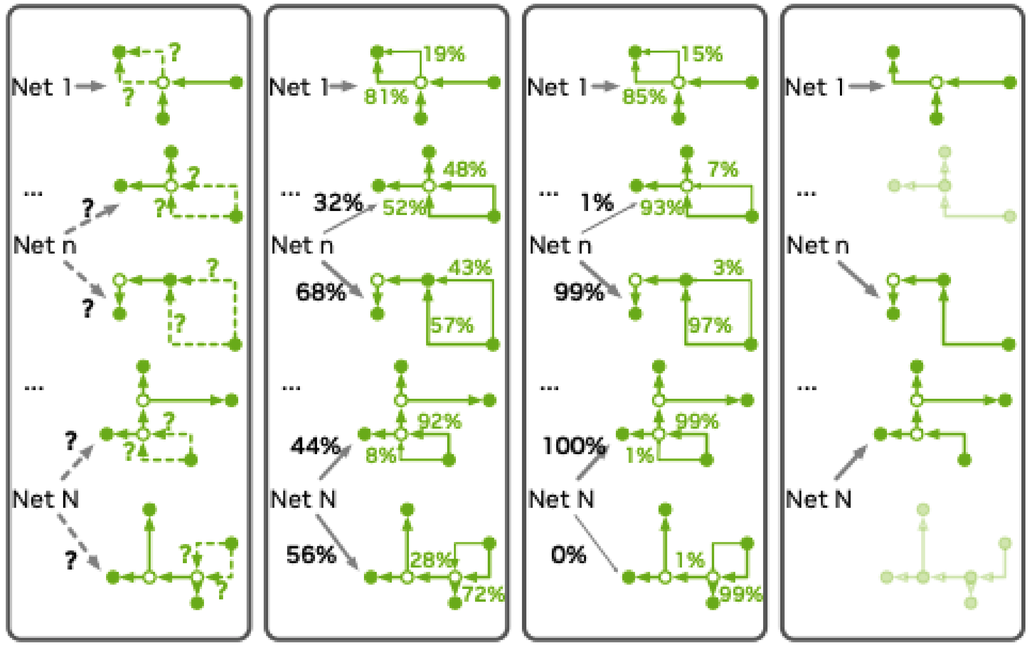
H. Ren et al, Nonlinear Placement for VLSI using TensorFlow on GPU, GTC'18;
Y. Lin et al, DREAMPlace, DAC '19

DGR – Differentiable Global Router

Relax the discrete global routing problem with a continuous gradient-based method

- Solve DAG 2D routing problem, originally solved with heuristic method
- Relax the discrete optimization problem to a continuous problem with gumbel_softmax function
- Create a cost function and leverage gradient-based optimization
- Derive the discrete solution from the continuous solution
- 1- 4% better on routed wirelength and via counts than SOTA academic routers
- Only one step in global routing is accelerated
  - Speedup over 100X on the DAG routing over ILP method
  - 20% over overall speedup due to quality improvement

Synthetic data parameters				Runtime (s)		Overflow			
Grid Graph	cap_e	Net #	box size	ILP	DGR	ILP	DGR*	DGR best	DGR worst
20×20	1	20	4	0.08	5.09	30	30	30	30
50×50	1	50	10	36.12	5.16	173	173	173	173
50×50	1	100	10	102.46	4.95	1028	1028	1028	1028
50×50	2	100	10	38.79	4.89	0	0	0	0
50×50	1	1000	10	4621.12	5.59	35445	35445	35445	35445
50×50	10	1000	10	2493.21	78.85	7407	7407	7407	7407
50×50	10	10000	10	N/A	81.24	N/A	364536	364536	364621
100×100	2	1000	20	5763.16	5.83	48846	48846	48849	48891
100×100	2	10000	20	N/A	75.90	N/A	785664	785664	785678
1000×1000	1	100000	200	N/A	924.23	N/A	80237614	80238208	80249124
			Ratio	>118.28	1.00	1.00	1.00	$>1-10^{-6}$	$>1-10^{-5}$



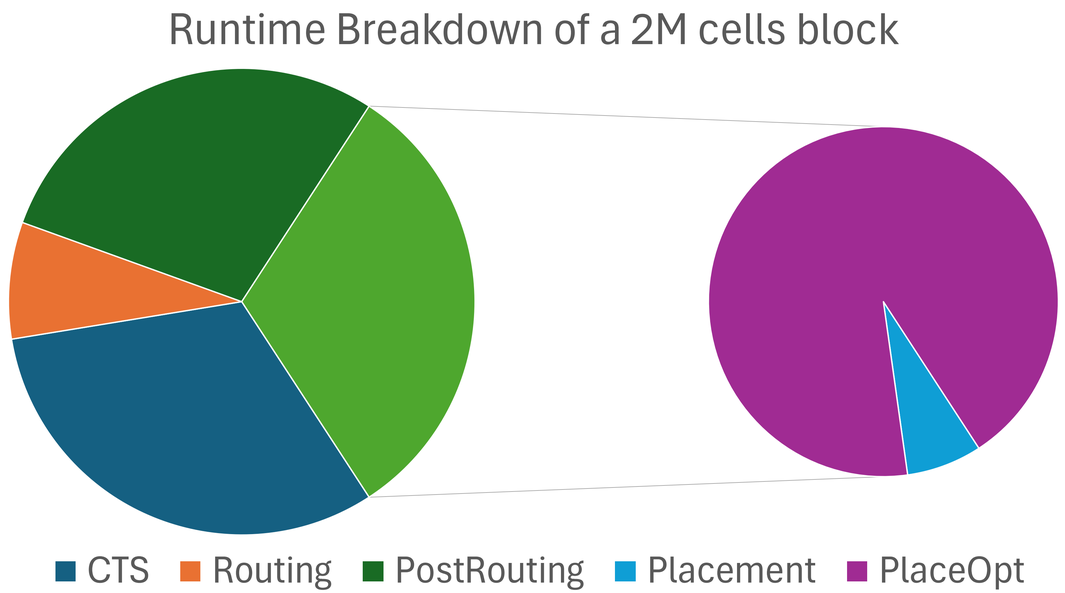
W. Li et al, DGR: Differentiable Global Router, DAC'24

Optimization is the Main PD Scalability Challenge

Amdahl's Law limits the scaling upper bound

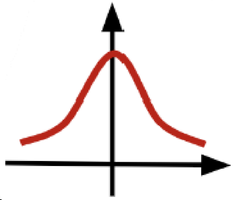
Optimization (gate sizing, buffering, etc. for better PPA) consumes the majority of the runtime in each major PD step

Why? Numerous heuristics to push for PPA limit, incremental timing, and routing updates



Generative Model For Optimization

Train a GenAI model with end-to-end optimized data



Auto Regressive (AR) Generative Model : $\mathrm{P}(x_{t+1}|x_1, x_2, \ldots, x_t)$

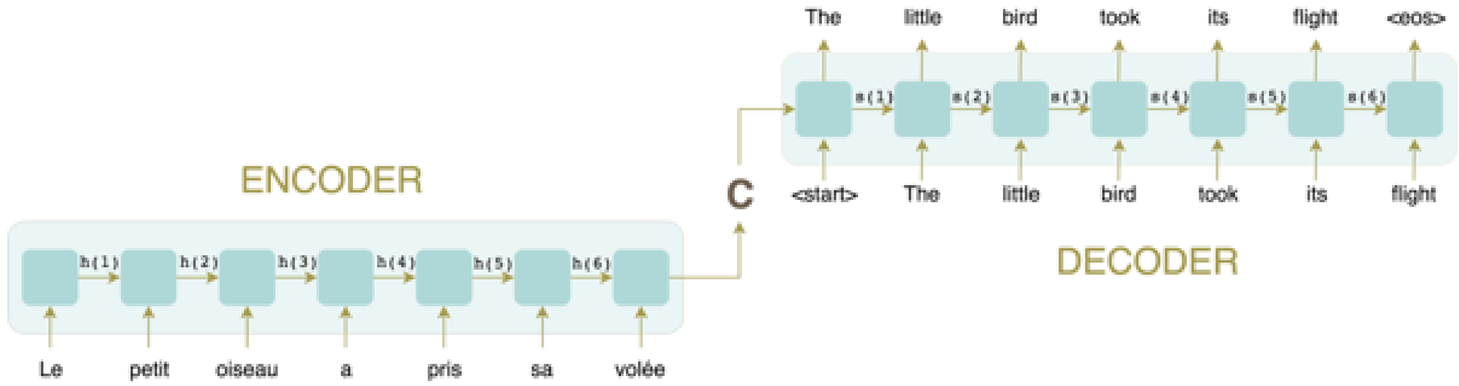
Consider the optimization variables as a sequence, generate optimized variable assignment a time

$$x_1 \rightarrow x_2|x_1 \rightarrow x_3|x_1, x_2 \ldots \rightarrow x_N|x_1, x_2, \ldots, x_{N-1}$$

Conditioned on problem configuration C

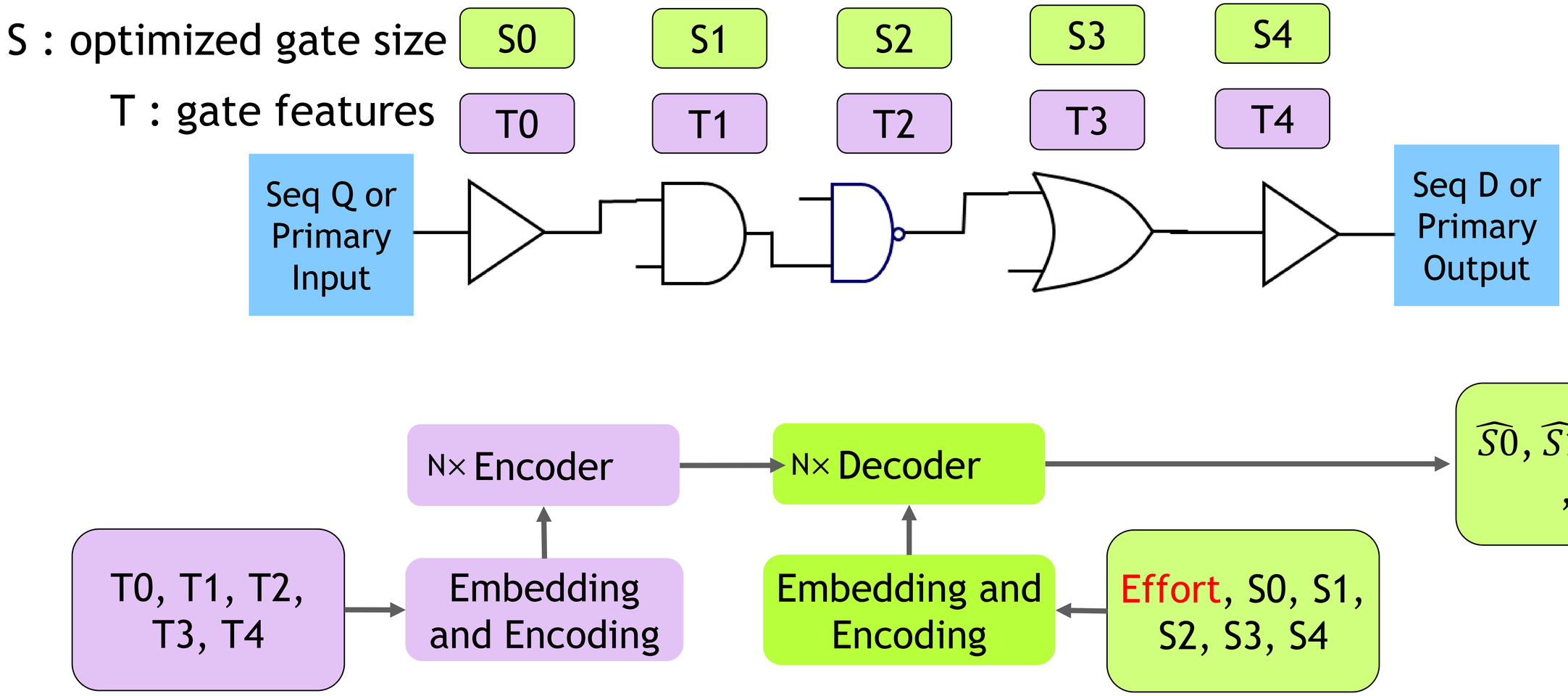
$$x_1|c \rightarrow x_2|x_1, c \rightarrow x_3|x_1, x_2, c \ldots \rightarrow x_N|x_1, x_2, \ldots, x_{N-1}, c$$

Transformer can be used as conditional generative model



TransSizer: Gate Sizing with Conditional AR model

End-to-end training circumvents Amdahl's Law



S. Nath et al. TransSizer: A Novel Transformer-Based Fast Gate Sizer, ICCAD'22

Optimization Results

Trained over a million timing paths from several 5nm designs

Accuracy is measured by comparing predicted sizes with true sizes： 88 – 92%

Compared to commercial tools, up to 1400X speedup with similar PPA

Generate initial condition alone can save big for runtime

Trained and inferenced on the same design, and results still not as good as commercial tools

Name	Method	WNS	TNS	NVE	Power	Area	Speedup
Des1	*Init*	-0.507	-23.01	71	0.303	171	–
	Tool	0.0	0.0	0	0.304	172	1
	Ours	-0.03	-0.399	25	0.304	172	125X
Des2	*Init*	-0.991	-2062.09	7421	68.22	92.2K	–
	Tool	-0.105	-54.71	2539	69.13	92.3K	1
	Ours	-0.386	-54.81	917	68.22	92.2K	50X
Des5	*Init*	-1.360	-9870.29	33341	287.97	173.5K	–
	Tool	-0.551	-295.64	2920	287.98	174K	1
	Ours	-0.467	-344.13	5413	288.09	174K	1440X

Comparing Gradient-based method with Generative AI

	Advantage	Issues
Gradient-based Method	• Scalable to millions of variables on GPUs • Grounded optimization	• Require differentiable objective functions • Require relaxation and approximation • **How to ensure global optimal**
Generative AI	• Scalable to millions of variables on GPUs • No need to compute objective function and its gradient • Optimum statistically	• Require a lot of optimized data • Data collection challenge • **How to deal with outliers**

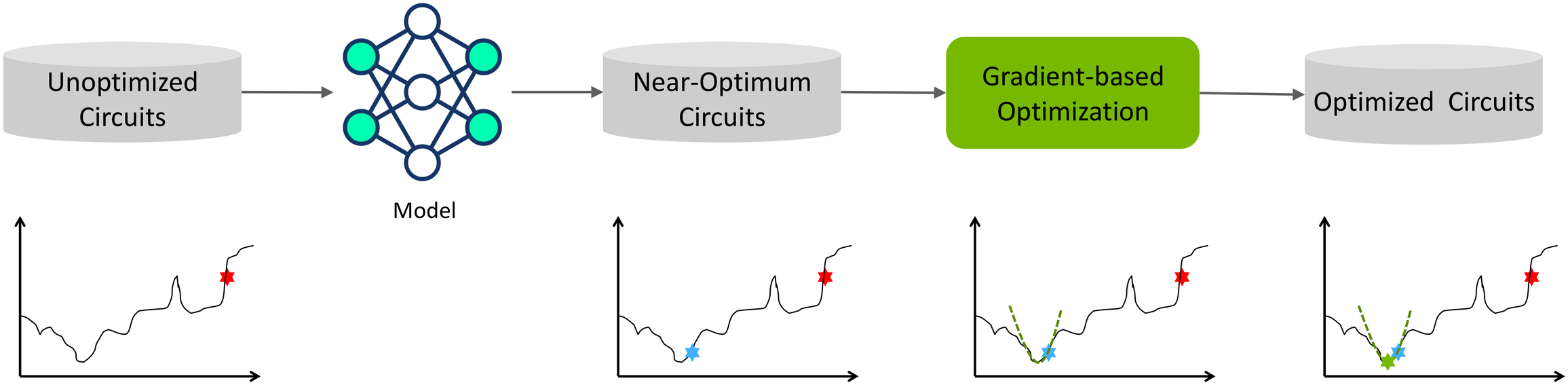
Combine Gen AI and Gradient-Based Method

Leverage Gen AI model to generate a solution that is close to the global optimum

Build gradient-based optimization in the global optimum neighborhood to search for the final solution

Need a gradient back from the timing loss function

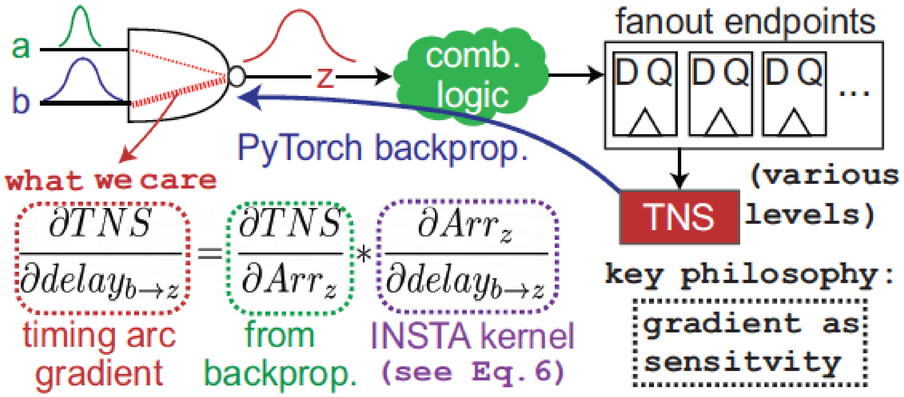
i.e. Sensitivity Analysis; e.g. sensitivity guided net weighting, TCAD'05

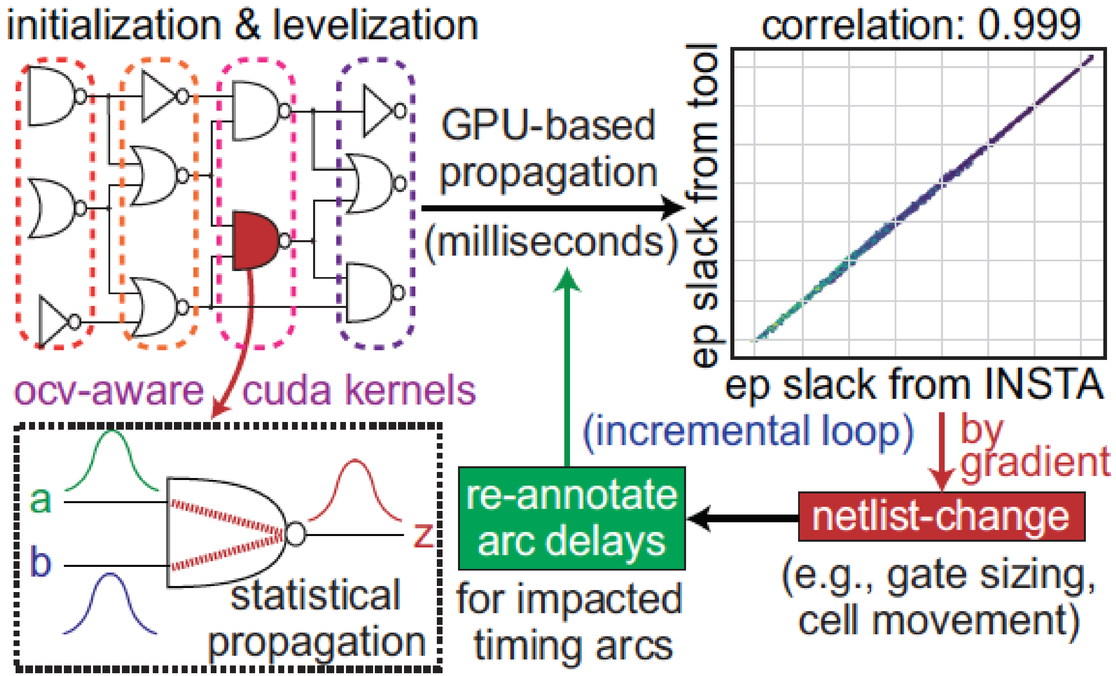


INSTA: GPU-Accelerated Differentiable Timer

Provide gradients to optimization

- Accelerate delay propagation part of STA on GPU
- Automatic gradient back with DL framework – leverage Log-Sum-Exponential (LSE) operator to smooth max operator
- Support statistical propagation and CPPR with top-k propagation
- Provide the main engine for differentiable optimization
  - Placement, routing, sizing, buffering, etc.
- 0.1 second on 4M cell/15M pin block with 0.999 correlation





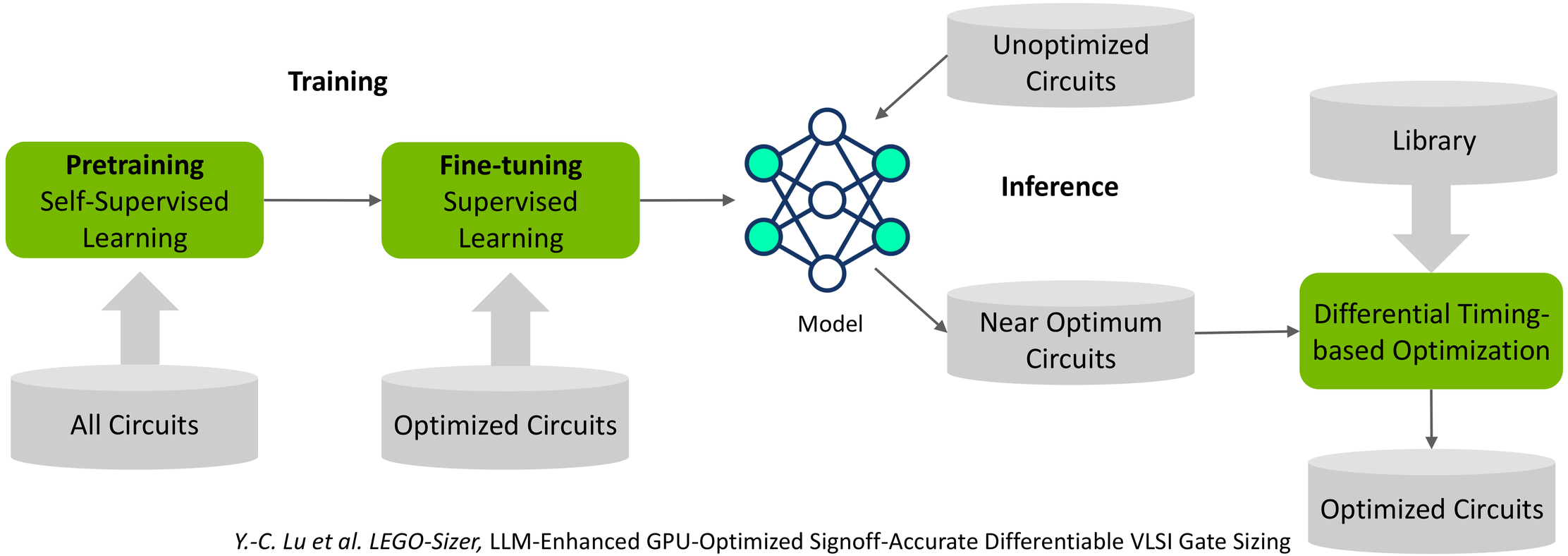
Y.-C. Lu et al, INSTA: An Ultra-Fast, Differentiable, Statistical Static Timing Analysis Engine for Industrial Physical Design Applications, DAC'25

LEGO-Sizer : 2nd attempt on GenAI for gate sizing

Encoder-only architecture predicts all gates on a path simultaneously

Similar to the pretraining/fine-tuning strategy, leverage unsupervised learning to learn the design representation and supervised learning to learn the optimization

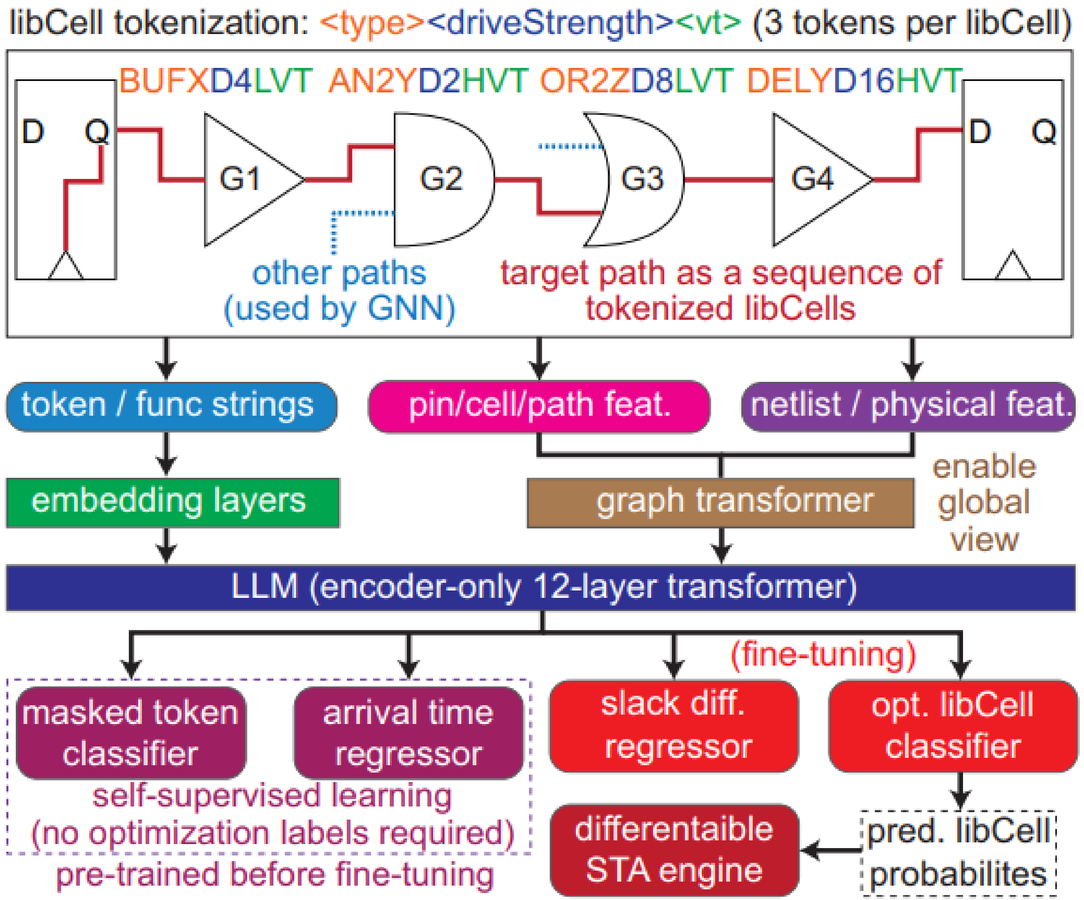
To fix the outliers after inference, run differential timing-based optimization



Y.-C. Lu et al. LEGO-Sizer, LLM-Enhanced GPU-Optimized Signoff-Accurate Differentiable VLSI Gate Sizing in Advanced Nodes, *ISPD'25*

LEGO-Sizer Architecture

- Sophisticated embeddings to capture gate and netlist features
  - LLM-inspired tokenization for each gate
  - Timing table generated gate embeddings
  - Graph transformer to capture netlist features
- Pretraining tasks:
  - Masked token classifier
  - Arrival time regressor
- Fine-tuning tasks:
  - Slack difference regressor
  - Optimized libCell classifier
- Inference returns the initial probability of each libCell
- Differentiable STA engine built on PyTorch to optimize the probability of each libCell after inference



LEGO-Sizer Results

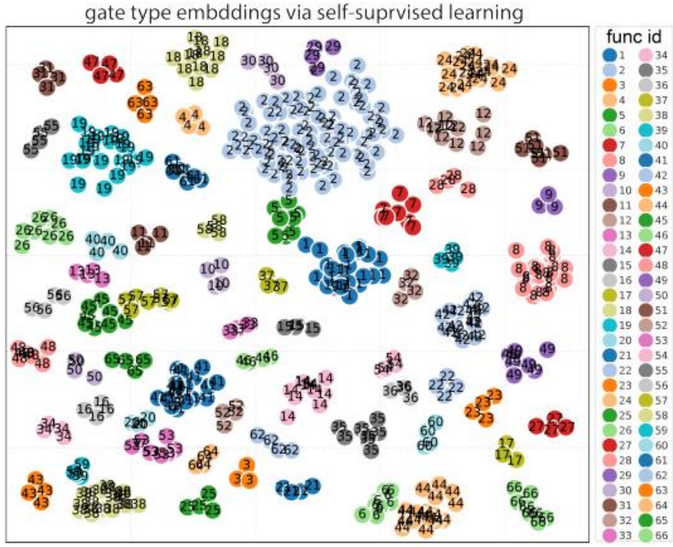
Pretrained on 28M timing paths over multiple 3nm blocks

Fine-tuned on ECO-optimized netlists

Inference on unseen designs from the fine-tuning

Better TNS/WNS optimization than ECO tools, >100X speedup

Learned embeddings show functional understanding



unseen design (# cells)	initial state (pre-optimized)				commercial signoff tool (32 threads)					LEGO-Size (GT + LLM + Differentiable STA)				
	WNS	TNS	#vio. EPs	total power	WNS	TNS (goal)	#vio. EPs	total power	speed-up	WNS	TNS (goal)	#vio. EPs	total power	speed-up
block1 (1.8M)	-0.066	-17.46	2410	238.6	-0.028	-1.953	271	238.8	1.00	-0.034	-1.237 (-36.7%)	184 (-47.3%)	238.8	125x
block2 (1.3M)	-0.041	-30.38	5410	293.4	-0.038	-7.82	1201	293.7	1.00	-0.035	-5.47 (-30.0%)	943 (-27.4%)	293.9	111x
block3 (1.2M)	-0.054	-10.03	1539	119.7	-0.032	-5.37	906	119.8	1.00	-0.033	-4.32 (-19.5%)	812 (-10.4%)	119.8	100x
block4 (1.5M)	-0.102	-59.25	6955	248.5	-0.097	-24.77	1912	249.1	1.00	-0.083	-20.36 (-17.8%)	1658 (-15.3%)	249.3	119x
block5 (1.4M)	-0.072	-11.33	1525	127.2	-0.038	-6.68	620	127.5	1.00	-0.036	-4.85 (-27.4%)	529 (-17.2%)	127.5	114x

Case Study

We only perform gradient descent on the cells that are predicted to be different by LLM

i.e., for cells whose argmax predictions are same as initial size, we do not perform GD on them

	Total	reg->reg	in->reg	reg->out	in->out
WNS	-0.834	-0.054	-0.206	-0.834	0.000
TNS	-1708.678	-10.026	-6.283	-1692.369	0.000
NUM	7393	1539	83	5771	0

initial

	Total	reg->reg	in->reg	reg->out	in->out
WNS	-0.834	-0.032	-0.204	-0.834	0.000
TNS	-1703.861	-5.374	-6.118	-1692.369	0.000
NUM	6758	906	81	5771	0

ECO Tool-optimized

Setup violations

	Total	reg->reg	in->reg	reg->out	in->out
WNS	-0.834	-0.035	-0.206	-0.834	0.000
TNS	-1705.311	-6.662	-6.280	-1692.369	0.000
NUM	7230	1376	83	5771	0

Model-only “predicted” @ F1 0.87

Setup violations

	Total	reg->reg	in->reg	reg->out	in->out
WNS	-0.834	-0.033	-0.205	-0.834	0.000
TNS	-1702.863	**-4.318**	-6.181	-1692.369	0.000
NUM	6666	**812**	83	5771	0

LEGO: Model + Gradient-based Optimization

Frameworks and Coding Assistant

Improve GPU code development efficiency

- DL frameworks such as PyTorch, JAX and general-purpose high level GPU program frameworks such as Numba, CuPy
  - Seamless GPU acceleration
  - High-level abstraction
  - Optimized performance
  - Support of mixed-precision
  - Rich visualization, profiling and distributed computing infra
  - Strong community support
- Coding assistants such as Cursor, CoPilot, Windsurf
  - Super good at C++ and Python, will get better at CUDA soon
  - Satya Nadella said 30% of MSFT code wrote by AI
  - Mark Zuckerberg said most Meta code will be written by AI in next 12-18 months
  - The majority of NVIDIA software engineers use coding assistants today

GPU-Accelerated EDA Opportunities

GPU is ready to become mainstream in EDA

ROI of investing in GPU-acceleration is ~~Low~~ High

- Limits of parallelism- EDA problems are not ‘embarrassingly parallel’
  - Tackle new applications that have ‘embarrassingly parallel’ characteristics
- Performance limitation – divergence, communication, and synchronization overheads
  - Leverage compiler/partition techniques to transform the problem
- Algorithm scalability – EDA algorithms are difficult to vectorize to scale on GPU
  - Use scalable algorithm: gradient-based optimization
- Discrete problems – EDA problems are discrete and difficult to scale
  - Relax discrete problems to continuous problems
- Limits of scaling – Amdahl’s Law
  - End-to-end training with Gen AI
  - Combining gradient-based methods with Gen AI
- Cost of development - CUDA code and maintaining two different versions of the tools
  - Leverage DL and other frameworks
  - Adopt a coding assistant

Acknowledgments

NVIDIA Design Automation Research Group

Haoxing (Mark) Ren

Team Leader

Chenhui Deng

Graph Learning for EDA, LLM for Chip Design

Chia-Tung (Mark) Ho

LLM Agent for Chip Design, Machine Learning for VLSI, DTCO, Standard Cell Layout Automation

Danny Liu

Reinforcement Learning, Machine Learning for EDA, Generative AI

Haoyu Yang

AI/LLM for EDA, GPU-accelerated EDA, AI for MultiPhysics Simulation, AI for Semiconductor Manufacturing

Rongjian Liang

Machine Learning for EDA, EDA for Machine Learning

Wen-Hao Liu

Circuits and VLSI Design

Yanqing Zhang

GPU Accelerated EDA, Machine Learning for EDA, GPU Accelerated Simulation

Yi-Chen Lu

Physical Design, Machine Learning for EDA

Yunsheng Bai

Formal Verification, Machine Learning for EDA

Also Contributions from Interns and Collaborators

To learn more about our research, check out https://research.nvidia.com/labs/electronic-design-automation/